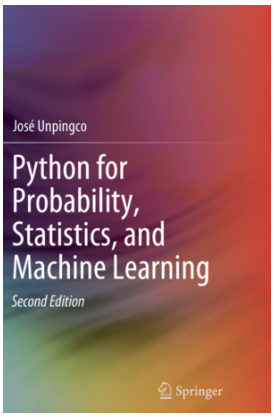
```
from IPython.display import Image
Image('../../Python_probability_statistics_machine_learning_2E.png',width=200)
```



Monte Carlo Sampling Methods

So far, we have studied analytical ways to transform random variables and how to augment these methods using Python. In spite of all this, we frequently must resort to purely numerical methods to solve real-world problems. Hopefully, now that we have seen the deeper theory, these numerical methods will feel more concrete. Suppose we want to generate samples of a given density, $f(x)$, given we already can generate samples from a uniform distribution, $\mathcal{U}[0,1]$. How do we know a random sample v comes from the $f(x)$ distribution? One approach is to look at how a histogram of samples of v approximates $f(x)$. Specifically,

$$\mathbb{P}(v \in N_\Delta(x)) = f(x)\Delta x \tag{1}$$

The histogram approximates the target probability density.

which says that the probability that a sample is in some N_Δ neighborhood of x is approximately $f(x)\Delta x$. Figure shows the target probability density function $f(x)$ and a histogram that approximates it. The histogram is generated from samples v. The hatched rectangle in the center illustrates Equation 1. The area of this rectangle is approximately $f(x)\Delta x$ where $x=0$, in this case. The width of the rectangle is $N_\Delta(x)$ The quality of the approximation may be clear visually, but to know that v samples are characterized by $f(x)$, we need the statement of Equation 1, which says that the proportion of samples v that fill the hatched rectangle is approximately equal to $f(x)\Delta x$.

Now that we know how to evaluate samples v that are characterized by the density $f(x)$, let's consider how to create these samples for both discrete and continuous random variables.

Inverse CDF Method for

Discrete Variables

Suppose we want to generate samples from a fair six-sided die. Our workhouse uniform random variable is defined continuously over the unit interval and the fair six-sided die is discrete. We must first create a mapping between the continuous random variable u and the discrete outcomes of the die. This mapping is shown in Figure where the unit interval is broken up into segments, each of length $1/6$. Each individual segment is assigned to one of the die outcomes. For example, if $u \in [1/6, 2/6)$, then the outcome for the die is 2. Because the die is fair, all segments on the unit interval are the same length. Thus, our new random variable v is derived from u by this assignment.

A uniform distribution random variable on the unit interval is assigned to the six outcomes of a fair die using these segements.

For example, for $v=2$, we have,

$$\mathbb{P}(v=2) = \mathbb{P}(u \in [1/6, 2/6)) = 1/6$$

where, in the language of the Equation 1, $f(x)=1$ (uniform distribution), $\Delta x = 1/6$, and $N_\Delta(2) = [1/6, 2/6)$. Naturally, this pattern holds for all the other die outcomes in $\{1,2,3,..,6\}$. Let's consider a quick simulation to make this concrete. The following code generates uniform random samples and stacks them in a Pandas dataframe.

```
import pandas as pd
import numpy as np
from pandas import DataFrame
```

```
u= np.random.rand(100)
df = DataFrame(data=u,columns=['u'])
```

The next block uses `pd.cut` to map the individual samples to the set $\{1, 2, \ldots, 6\}$ labeled `v`.

```
labels = [1,2,3,4,5,6]
df['v']=pd.cut(df.u,np.linspace(0,1,7),
               include_lowest=True,labels=labels)
```

This is what the dataframe contains. The `v` column contains the samples drawn from the fair die.

```
df.head()
```

	u	v
0	0.596534	4
1	0.410969	3
2	0.416233	3
3	0.171567	2
4	0.716067	5

The following is a count of the number of samples in each group. There should be roughly the same number of samples in each group because the die is fair.

```
df.groupby('v').count()
```

	u
v	
1	11
2	19
3	18
4	17
5	16
6	19

So far, so good. We now have a way to simulate a fair die from a uniformly distributed random variable.

To extend this to unfair die, we need only make some small adjustments to this code. For example, suppose that we want an unfair die so that $\mathbb{P}(1) = \mathbb{P}(2) = \mathbb{P}(3) = 1/12$ and $\mathbb{P}(4) = \mathbb{P}(5) = \mathbb{P}(6) = 1/4$. The only change we have to make is with `pd.cut` as follows,

```
df['v']=pd.cut(df.u,[0,1/12,2/12,3/12,2/4,3/4,1],
               include_lowest=True,labels=labels)
df.groupby('v').count()/df.shape[0]
```

	u
v	
1	0.06
2	0.05
3	0.10
4	0.27
5	0.24
6	0.28

where now these are the individual probabilities of each digit. You can take more than `100` samples to get a clearer view of the individual probabilities but the mechanism for generating them is the same. The method is called the inverse CDF [^CDF] method because the CDF

(namely, `[0,1/12,2/12,3/12,2/4,3/4,1]`) in the last example has been inverted (using the `pd.cut` method) to generate the samples.
The inversion is easier to see for continuous variables, which we consider next.

[^CDF]: Cumulative density function. Namely, $F(x) = \mathbb{P}(X < x)$.

Inverse CDF

Method for Continuous Variables

The method above applies to continuous random variables, but now we have to squeeze the intervals down to individual points. In the example above, our inverse function was a piecewise function that operated on uniform random samples. In this case, the piecewise function collapses to a continuous inverse function. We want to generate random samples for a CDF that is invertible. As before, the criterion for generating an appropriate sample v is the following,

$$\mathbb{P}(F(x) < v < F(x+\Delta x)) = F(x+\Delta x) - F(x) = \int_x^{x+\Delta x} f(u)du \approx f(x)\Delta x$$

which says that the probability that the sample v is contained in a Δx interval is approximately equal to $f(x)\Delta x$, at that point. Once again, the trick is to use a uniform random sample u and an invertible CDF $F(x)$ to construct these samples. Note that for a uniform random variable $u \sim \mathcal{U}[0,1]$, we have,

$$\begin{aligned}\mathbb{P}(x < F^{-1}(u) < x+\Delta x) &= \mathbb{P}(F(x) < u < F(x+\Delta x)) \\ &= F(x+\Delta x) - F(x) \\ &= \int_x^{x+\Delta x} f(p)dp \approx f(x)\Delta x\end{aligned}$$

This means that $v = F^{-1}(u)$ is distributed according to $f(x)$, which is what we want.

Let's try this to generate samples from the exponential distribution,

$$f_\alpha(x) = \alpha e^{-\alpha x}$$

which has the following CDF,

$$F(x) = 1 - e^{-\alpha x}$$

and corresponding inverse,

$$F^{-1}(u) = \frac{1}{\alpha}\ln\frac{1}{(1-u)}$$

Now, all we have to do is generate some uniformly distributed random samples and then feed them into F^{-1}.

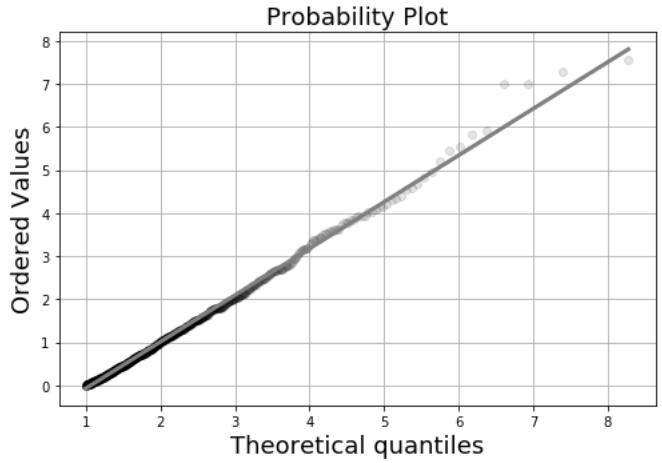
```
from numpy import array, log
import scipy.stats
alpha = 1.   # distribution parameter
nsamp = 1000 # num of samples
# define uniform random variable
u=scipy.stats.uniform(0,1)
# define inverse function
Finv=lambda u: 1/alpha*log(1/(1-u))
# apply inverse function to samples
v = array(list(map(Finv,u.rvs(nsamp))))
```

Now, we have the samples from the exponential distribution, but how do we know the method is correct with samples distributed accordingly? Fortunately, `scipy.stats` already has a exponential distribution, so we can check our work against the reference using a *probability plot* (i.e., also known as a *quantile-quantile* plot). The following code sets up the probability plot from `scipy.stats`.

```
%matplotlib inline

from matplotlib.pylab import setp, subplots
fig,ax = subplots()
fig.set_size_inches((7,5))
_=scipy.stats.probplot(v,(1,),dist='expon',plot=ax)
line=ax.get_lines()[0]
_=setp(line,'color','k')
_=setp(line,'alpha',.1)
line=ax.get_lines()[1]
_=setp(line,'color','gray')
_=setp(line,'lw',3.0)
```

```
_=setp(ax.yaxis.get_label(),'fontsize',18)
_=setp(ax.xaxis.get_label(),'fontsize',18)
_=ax.set_title('Probability Plot',fontsize=18)
_=ax.grid()
fig.tight_layout()
fig.savefig('fig-probability/Sampling_Monte_Carlo_005.png')
```




```
fig,ax=subplots()
scipy.stats.probplot(v,(1,),dist='expon',plot=ax)
```

Note that we have to supply an axes object (`ax`) for it to draw on. The result is Figure. The more the samples line match the diagonal line, the more they match the reference distribution (i.e., exponential distribution in this case). You may also want to try `dist=norm` in the code above To see what happens when the normal distribution is the reference distribution.

The samples created using the inverse cdf method match the exponential reference distribution.

Rejection Method

In some cases, inverting the CDF may be impossible. The *rejection* method can handle this situation. The idea is to pick two uniform random variables $u_1, u_2 \sim \mathcal{U}[a,b]$ so that

$$\mathbb{P}\left(u_1 \in N_\Delta(x) \bigwedge u_2 < \frac{f(u_1)}{M}\right) \approx \frac{\Delta x}{b-a}\frac{f(u_1)}{M}$$

where we take $x = u_1$ and $f(x) < M$. This is a two-step process. First, draw u_1 uniformly from the interval $[a,b]$. Second, feed it into $f(x)$ and if $u_2 < f(u_1)/M$, then you have a valid sample for $f(x)$. Thus, u_1 is the proposed sample from f that may or may not be rejected depending on u_2. The only job of the M constant is to scale down the $f(x)$ so that the u_2 variable can span the range. The *efficiency* of this method is the probability of accepting u_1 which comes from integrating out the above approximation,

$$\int \frac{f(x)}{M(b-a)}dx = \frac{1}{M(b-a)}\int f(x)dx = \frac{1}{M(b-a)}$$

This means that we don't want an unecessarily large M because that makes it more likely that samples will be discarded.

Let's try this method for a density that does not have a continuous inverse [^normalization]. [^normalization]: Note that this example density does not *exactly* integrate out to one like a probability density function should, but the normalization constant for this is distracting for our purposes here.

$$f(x) = \exp\left(-\frac{(x-1)^2}{2x}\right)(x+1)/12$$

where $x > 0$. The following code implements the rejection plan.

```
import numpy as np
x = np.linspace(0.001,15,100)
f= lambda x: np.exp(-(x-1)**2/2./x)*(x+1)/12.
fx = f(x)
M=0.3                          # scale factor
u1 = np.random.rand(10000)*15  # uniform random samples scaled out
u2 = np.random.rand(10000)     # uniform random samples
idx,= np.where(u2<=f(u1)/M)    # rejection criterion
```

```
v = u1[idx]
```

```
fig,ax=subplots()
fig.set_size_inches((9,5))
_=ax.hist(v,density=True,bins=40,alpha=.3,color='gray')
_=ax.plot(x,fx,'k',lw=3.,label='$f(x)$')
_=ax.set_title('Estimated Efficency=%3.1f%%'%(100*len(v)/len(u1)),
               fontsize=18)
_=ax.legend(fontsize=18)
_=ax.set_xlabel('$x$',fontsize=24)
fig.tight_layout()
fig.savefig('fig-probability/Sampling_Monte_Carlo_007.png')
```

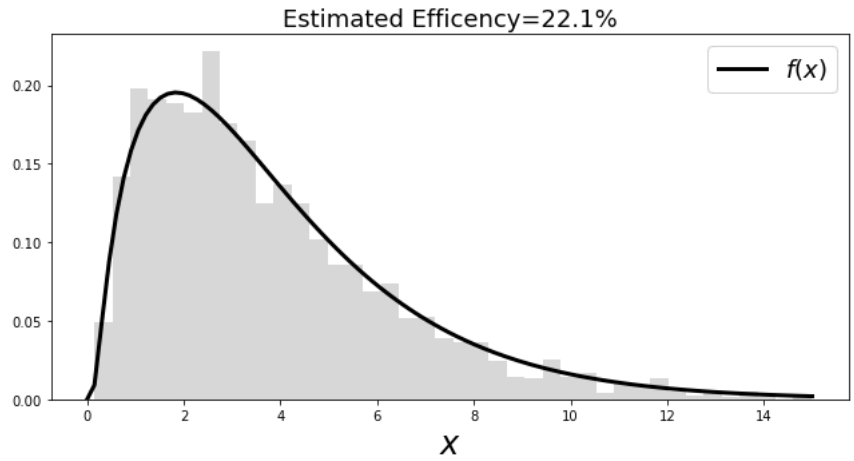
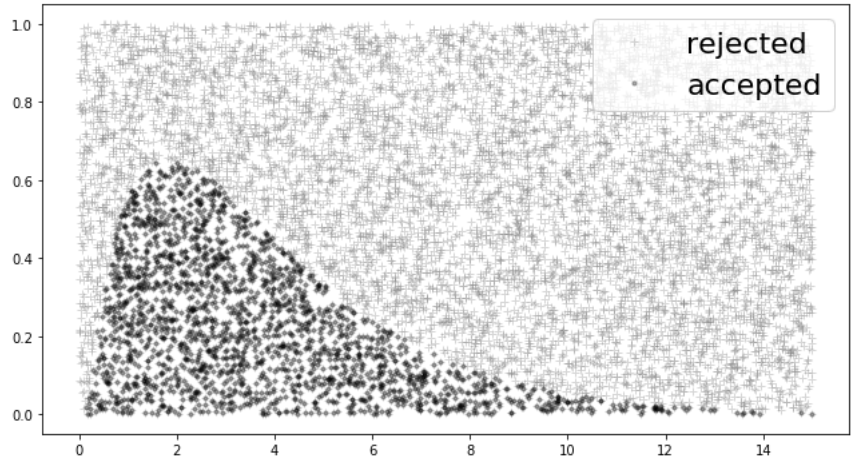


Figure shows a histogram of the so-generated samples that nicely fits the probability density function. The title in the figure shows the efficiency (the number of rejected samples), which is poor. It means that we threw away most of the proposed samples. Thus, even though there is nothing conceptually wrong with this result, the low efficiency must be fixed, as a practical matter. Figure shows where the proposed samples were rejected. Samples under the curve were retained (i.e., $u_2 < \frac{f(u_1)}{M}$) but the vast majority of the samples are outside this umbrella.

The rejection method generate samples in the histogram that nicely match the target distribution. Unfortunately, the efficiency is not so good.

```
fig,ax=subplots()
fig.set_size_inches((9,5))
_=ax.plot(u1,u2,'+',label='rejected',alpha=.3,color='gray')
_=ax.plot(u1[idx],u2[idx],'.',label='accepted',alpha=.3,color='k')
_=ax.legend(fontsize=22)
fig.tight_layout()
fig.savefig('fig-probability/Sampling_Monte_Carlo_008.png')
```



The proposed samples under the curve were accepted and the others were not. This shows the majority of samples were rejected.

The rejection method uses u_1 to select along the domain of $f(x)$ and the other u_2 uniform random variable decides whether to accept or not. One idea would be to choose u_1 so that x values are coincidentally those that are near the peak of $f(x)$, instead of uniformly anywhere in the domain, especially near the tails, which are low probability anyway. Now, the trick is to find a new density function $g(x)$ to sample from that has a similiar concentration of probability density. One way it to familiarize oneself with the probability density functions that have adjustable parameters and fast random sample generators already. There are lots of places to look and, chances are, there is likely already such a generator for your problem. Otherwise, the family of β densities is a good place to start. To be explicit, what we want is $u_1 \sim g(x)$ so that, returning to our earlier argument,

$$\mathbb{P}\left(u_1 \in N_\Delta(x) \bigwedge u_2 < \frac{f(u_1)}{M}\right) \approx g(x)\Delta x \frac{f(u_1)}{M}$$

but this is *not* what we need here. The problem is with the second part of the logical $\bigwedge$ conjunction. We need to put something there that will give us something proportional to $f(x)$. Let us define the following,

$$h(x) = \frac{f(x)}{g(x)} \tag{2}$$

with corresponding maximum on the domain as $h_{\max}$ and then go back and construct the second part of the clause as

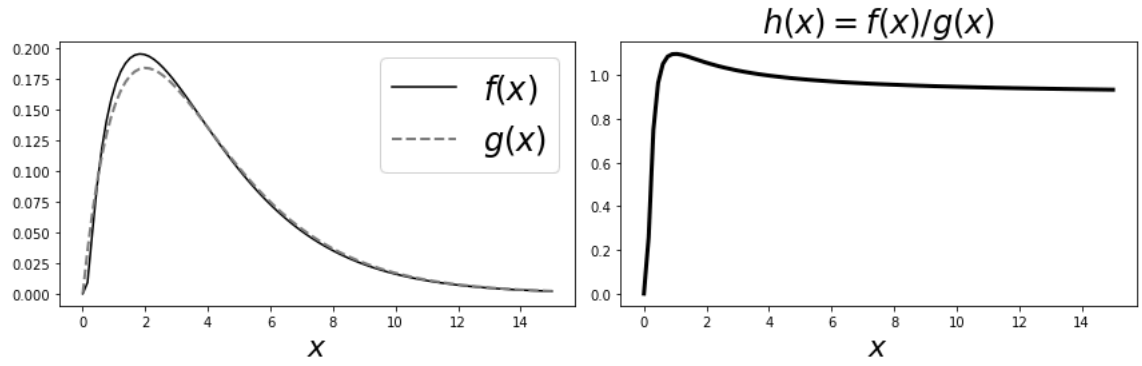
$$\mathbb{P}\left(u_1 \in N_\Delta(x) \bigwedge u_2 < \frac{h(u_1)}{h_{\max}}\right) \approx g(x)\Delta x \frac{h(u_1)}{h_{\max}} = f(x)/h_{\max}$$

Recall that satisfying this criterion means that $u_1 = x$. As before, we can estimate the probability of acceptance of the u_1 as $1/h_{\max}$.

Now, how to construct the $g(x)$ function in the denominator of Equation 2? Here's where familarity with some standard probability densities pays off. For this case, we choose the χ^2 distribution. The following plots the $g(x)$ and $f(x)$ (left plot) and the corresponding $h(x) = f(x)/g(x)$ (right plot). Note that $g(x)$ and $f(x)$ have peaks that almost coincide, which is what we are looking for.

```
ch=scipy.stats.chi2(4) # chi-squared
h = lambda x: f(x)/ch.pdf(x) # h-function
```

```
fig,axs=subplots(1,2,sharex=True)
fig.set_size_inches(12,4)
_=axs[0].plot(x,fx,label='$f(x)$',color='k')
_=axs[0].plot(x,ch.pdf(x),'--',lw=2,label='$g(x)$',color='gray')
_=axs[0].legend(loc=0,fontsize=24)
_=axs[0].set_xlabel(r'$x$',fontsize=22)
_=axs[1].plot(x,h(x),'-k',lw=3)
_=axs[1].set_title('$h(x)=f(x)/g(x)$',fontsize=24)
_=axs[1].set_xlabel(r'$x$',fontsize=22)
fig.tight_layout()
fig.savefig('fig-probability/Sampling_Monte_Carlo_009.png')
```

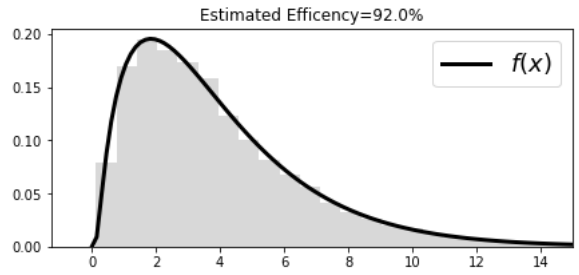


The plot on the right shows $h(x) = f(x)/g(x)$ and the one on the left shows $f(x)$ and $g(x)$ separately.

Now, let's generate some samples from this χ^2 distribution with the rejection method.

```
hmax=h(x).max()
u1 = ch.rvs(5000)        # samples from chi-square distribution
u2 = np.random.rand(5000)# uniform random samples
idx = (u2 <= h(u1)/hmax) # rejection criterion
v = u1[idx]              # keep these only
```

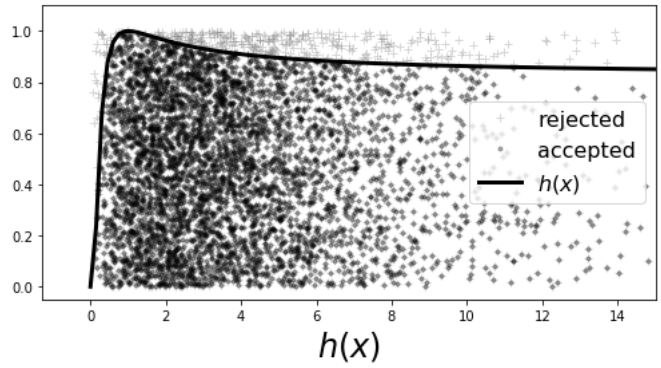
```
fig,ax=subplots()
fig.set_size_inches((7,3))
_=ax.hist(v,density=True,bins=40,alpha=.3,color='gray')
_=ax.plot(x,fx,color='k',lw=3.,label='$f(x)$')
_=ax.set_title('Estimated Efficency=%3.1f%%'%(100*len(v)/len(u1)))
_=ax.axis(xmax=15)
_=ax.legend(fontsize=18)
fig.savefig('fig-probability/Sampling_Monte_Carlo_010.png')
```



Using the updated method, the histogram matches the target probability density function with high efficiency.

Using the χ^2 distribution with the rejection method results in throwing away less than 10% of the generated samples compared with our prior example where we threw out at least 80%. This is dramatically more efficient! Figure shows that the histogram and the probability density function match. For completeness, Figure shows the samples with the corresponding threshold $h(x)/h_{\max}$ that was used to select them.

```
fig,ax=subplots()
fig.set_size_inches((7,4))
_=ax.plot(u1,u2,'+',label='rejected',alpha=.3,color='gray')
_=ax.plot(u1[idx],u2[idx],'g.',label='accepted',alpha=.3,color='k')
_=ax.plot(x,h(x)/hmax,color='k',lw=3.,label='$h(x)$')
_=ax.legend(fontsize=16,loc=0)
_=ax.set_xlabel('$x$',fontsize=24)
_=ax.set_xlabel('$h(x)$',fontsize=24)
_=ax.axis(xmax=15,ymax=1.1)
fig.tight_layout()
fig.savefig('fig-probability/Sampling_Monte_Carlo_011.png')
```



Fewer proposed points were rejected in this case, which means better efficiency.

Sampling Importance Resampling

An alternative to the Rejection Method that does not involve rejecting samples or coming up with M bounds or bounding functions is the Sampling Importance Resampling (SIR) method. Choose a tractable g probability density function and draw a n samples from it, $\{x_i\}_{i=1}^n$. Our objective is to derive samples f. Next, compute the following,

$$q_i = \frac{w_i}{\sum w_i}$$

where

$$w_i = \frac{f(x_i)}{g(x_i)}$$

The q_i define a probability mass function whose samples approximate samples from f. To see this, consider,

$$\begin{aligned}\mathbb{P}(X \leq a) &= \sum_{i=1}^{n} q_i \mathbb{I}_{(-\infty,a]}(x_i) \\ &= \frac{\sum_{i=1}^{n} w_i \mathbb{I}_{(-\infty,a]}(x_i)}{\sum_{i=1}^{n} w_i} \\ &= \frac{\frac{1}{n}\sum_{i=1}^{n} \frac{f(x_i)}{g(x_i)} \mathbb{I}_{(-\infty,a]}(x_i)}{\frac{1}{n}\sum_{i=1}^{n} \frac{f(x_i)}{g(x_i)}}\end{aligned}$$

Because the samples are generated from the g probability distribution, the numerator is approximately,

$$\mathbb{E}_g\left(\frac{f(x)}{g(x)}\right) = \int_{-\infty}^{a} f(x)dx$$

which gives

$$\mathbb{P}(X \leq a) = \int_{-\infty}^{a} f(x)dx$$

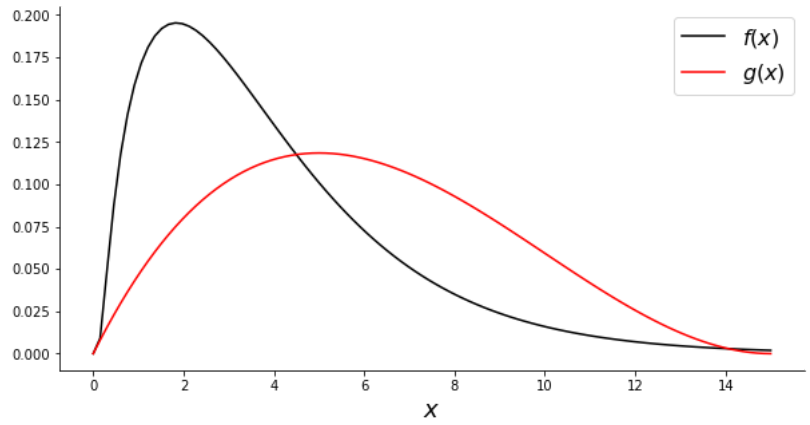
which shows that the samples generated this way are f-distributed. Note more samples have to be generated from this probability mass function the further away g is from the desired function f. Further, because there is no rejection step, we no longer have the issue of efficiency.

For example, let us choose a beta distribution for g, as in the following code

```
g = scipy.stats.beta(2,3)
```

This distribution does not bear a strong resemblance to our desired f function from last section. as shown in the Figure below. Note that we scaled the domain of the beta distribution to get it close to the support of f.

```
fig,ax=subplots(figsize=[10,5])
g = scipy.stats.beta(2,3)
_=ax.plot(x,fx,label='$f(x)$',color='k')
_=ax.plot(np.linspace(0,1,100)*15,1/15*g.pdf(np.linspace(0,1,100)),label='$g(x)$',color='r')
_=ax.legend(fontsize=16)
_=ax.set_xlabel('$x$',fontsize=18)
ax.spines['right'].set_visible(False)
ax.spines['top'].set_visible(False)
fig.savefig('fig-probability/Sampling_Monte_Carlo_012.png')
```



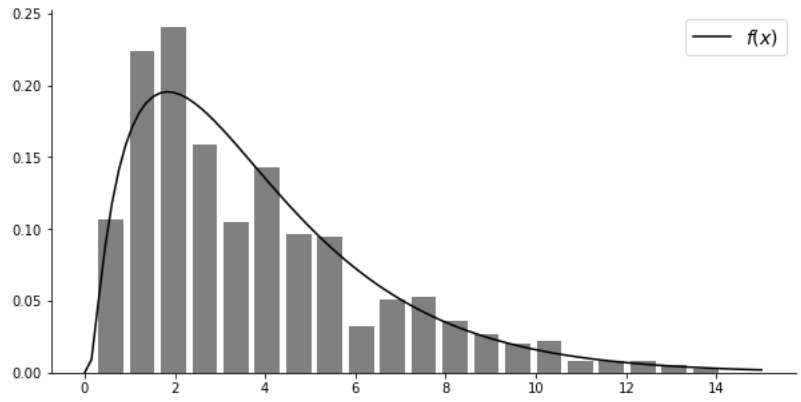
Histogram of samples generated using SIR comparted to target probability density function.

In the next block, we sample from the g distribution and compute the weights as described above. The final step is to sample from this new probability mass function. The resulting normalized histogram is shown compared to the target f probability density function in Figure.

```
xi = g.rvs(500)
w = np.array([f(i*15)/g.pdf(i) for i in xi])
```

```
fsamples=np.random.choice(xi*15,5000,p = w/w.sum())
```

```
fig,ax=subplots(figsize=[10,5])
_=ax.hist(fsamples,rwidth=.8,density=True,bins=20,color='gray')
_=ax.plot(x,fx,label='$f(x)$',color='k')
_=ax.legend(fontsize=14)
ax.spines['right'].set_visible(False)
ax.spines['top'].set_visible(False)
fig.savefig('fig-probability/Sampling_Monte_Carlo_013.png')
```



Histogram and probability density function using SIR.

In this section, we investigated how to generate random samples from a given distribution, beit discrete or continuous. For the continuous case, the key issue was whether or not the cumulative density function had a continuous inverse. If not, we had to turn to the rejection method, and find an appropriate related density that we could easily sample from to use as part of a rejection threshold. Finding such a function is an art, but many families of probability densities have been studied over the years that already have fast random number generators.

The rejection method has many complicated extensions that involve careful partitioning of the domains and lots of special methods for corner cases. Nonetheless, all of these advanced techniques are still variations on the same fundamental theme we illustrated here [dunn2011exploring], [johnson1995continuous].